

Documentation

Testing Examples of the Deep BSDE Solver

Patrick Streeb

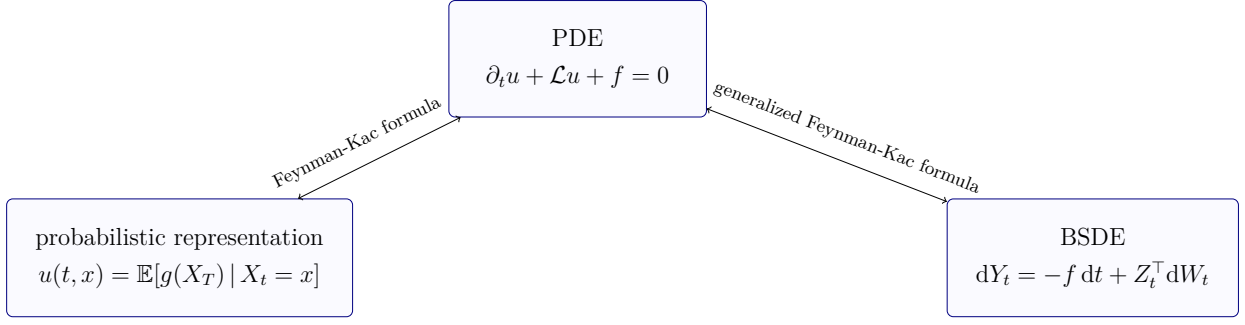
August 1, 2026

Contents

1	Introduction	1
2	Preliminaries	2
3	The Deep BSDE Method	4
4	Implementation	6
5	Examples	7
5.1	Hamilton-Jacobi-Bellman Equation	7
5.2	Allen-Cahn Equation	10
5.3	Nonlinear Black-Scholes Equation with Default Risk	11
5.4	Reaction-Diffusion Equation	13
5.5	Stability in the Starting Value	14
6	Conclusion	15

1 Introduction

Grid-based numerical methods for partial differential equations have a computational cost that grows exponentially with the dimension d , which is known as the curse of dimensionality. Nevertheless, many practical applications lead to high-dimensional problems. Monte Carlo methods applied to stochastic representations of PDE solutions work well in many cases, but perform less effectively when nested Monte Carlo simulation is required, since the computational cost does not scale linearly with the dimension of the problem. A second approach is the deep BSDE method of Han, Jentzen, and E [1], which indeed overcomes the curse of dimensionality. By reformulating a large class of PDEs as backward stochastic differential equations (BSDEs) and discretizing these equations, the resulting recursion contains two unknowns. Time-dependent neural networks are therefore used to approximate one of them. Increasing the dimension of the state only adds further components to the recursion, which can be evaluated in parallel. The article by Han, Jentzen, and E [1] is our main reference, and we refer to it as the reference paper or, for short, the paper.



numerical methods

Monte Carlo method

finite differences

deep BSDE solver

Figure 1: The three formulations of a parabolic terminal value problem and their numerical methods.

The goal of this documentation is to reproduce the examples in the paper, where the approach was introduced first. To try out our own architectures, we adapted the solver architecture to our own version. We could sometimes yield better results, than the ones postulated in the paper. Overall, the results are reproducible and we can confirm them for the presented examples. All source code developed for this project is available in the accompanying GitHub repository.¹ In total, there were four examples investigated.

- (1) For the Hamilton-Jacobi-Bellman equation of a linear-quadratic control problem the trained value matches the Monte Carlo reference with a relative error of 0.01 % (paper 0.17 %).
- (2) For the Allen-Cahn equation the error against the branching-diffusion value is 0.02 % (paper 0.30 %).
- (3) For the nonlinear Black-Scholes equation with default risk the error against the multilevel Picard value is 0.45 % (paper 0.46 %).
- (4) The final example was a reaction-diffusion equation, where we showed, that the accuracy improves with the number of hidden layers per subnetwork. With the 40000 epochs of the paper the error drops to 0.93 % at four hidden layers, against 0.53 % in the paper.

2 Preliminaries

We fix a finite horizon $T \in (0, \infty)$ and a filtered probability space $(\Omega, \mathcal{F}, \mathbb{P})$ with filtration $(\mathcal{F}_t)_{t \in [0, T]}$ satisfying the usual conditions and carrying a d -dimensional standard Brownian motion $W = (W_t)_{t \in [0, T]}$. Every expectation in this documentation refers to this space, we write $\mathbb{E}$ for $\mathbb{E}^{\mathbb{P}}$ and $\mathbb{E}[\cdot \mid \mathcal{F}_t]$ for the conditional expectation given $\mathcal{F}_t$. Let

$$\mu: [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad \sigma: [0, T] \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$$

be measurable. Suppose that there exists a constant $L > 0$ such that, for all $t \in [0, T]$ and $x, y \in \mathbb{R}^d$,

$$\begin{aligned} |\mu(t, x) - \mu(t, y)| + \|\sigma(t, x) - \sigma(t, y)\| &\leq L|x - y| \\ |\mu(t, x)| + \|\sigma(t, x)\| &\leq L(1 + |x|). \end{aligned}$$

¹<https://github.com/patrickstreeb/deep-bsde-solver-examples>

Here, $\|\cdot\|$ denotes the Frobenius norm and $I_d \in \mathbb{R}^{d \times d}$ the d -dimensional identity matrix. For each $(t, x) \in [0, T] \times \mathbb{R}^d$, let $X^{t,x} = (X_s^{t,x})_{s \in [t, T]}$ denote the unique strong solution of

$$X_s^{t,x} = x + \int_t^s \mu(r, X_r^{t,x}) dr + \int_t^s \sigma(r, X_r^{t,x}) dW_r, \quad s \in [t, T].$$

The stochastic integral is understood in the Itô sense. The classical Feynman–Kac formula represents solutions of linear parabolic terminal value problems as expectations; see [5, Chapter 8]. We denote by $\mathcal{H}_x u = (\partial_{x_i x_j} u)_{i,j=1}^d$ the Hessian of u with respect to the spatial variable.

Theorem 2.1: Feynman–Kac Formula

Let $g: \mathbb{R}^d \rightarrow \mathbb{R}$ be continuous, and let $u \in C([0, T] \times \mathbb{R}^d) \cap C^{1,2}([0, T) \times \mathbb{R}^d)$ satisfy

$$\begin{aligned} \partial_t u(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^\top(t, x) \mathcal{H}_x u(t, x)) + \mu(t, x) \cdot \nabla_x u(t, x) &= 0 \\ u(T, x) &= g(x) \end{aligned}$$

for $(t, x) \in [0, T) \times \mathbb{R}^d$. Assume that there exists a constant $C > 0$ such that

$$|u(t, x)| + |\sigma^\top(t, x) \nabla_x u(t, x)| \leq C(1 + |x|)$$

for every $(t, x) \in [0, T) \times \mathbb{R}^d$. Then

$$u(t, x) = \mathbb{E} [g(X_T^{t,x})], \quad (t, x) \in [0, T) \times \mathbb{R}^d.$$

Proof. See [5, Chapter 8]. □

If we for example take $d = 1$, $\mu = 0$, $\sigma = 1$, and $g(x) = x^2$ we get

$$X_s^{t,x} = x + W_s - W_t, \quad s \in [t, T],$$

and therefore

$$X_T^{t,x} = x + W_T - W_t.$$

We can now apply the Theorem of Feynman and Kac. Since W is a Brownian motion, we have $W_T - W_t \sim \mathcal{N}(0, T - t)$ by definition and since it lives on $(\Omega, \mathcal{F}, \mathbb{P})$, the expectation of the increment under $\mathbb{P}$ is zero and the squared increment is equal to $T - t$. We get

$$\begin{aligned} u(t, x) &= \mathbb{E} [g(X_T^{t,x})] \\ &= \mathbb{E} [(x + W_T - W_t)^2] \\ &= x^2 + 2x \mathbb{E} [W_T - W_t] + \mathbb{E} [(W_T - W_t)^2] \\ &= x^2 + T - t. \end{aligned}$$

The generalization to semilinear equations replaces the conditional expectation by a pair of processes solving a backward stochastic differential equation. We first fix the class of equations.

Definition 2.2: Semilinear Parabolic Terminal Value Problem

Let $f: [0, T] \times \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}^d \rightarrow \mathbb{R}$ and $g: \mathbb{R}^d \rightarrow \mathbb{R}$ be measurable. A *semilinear parabolic terminal value problem* is the problem of finding $u \in C^{1,2}([0, T] \times \mathbb{R}^d)$ with

$$\begin{aligned} \partial_t u(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^\top(t, x) \mathcal{H}_x u(t, x)) \\ + \nabla u(t, x) \cdot \mu(t, x) + f(t, x, u(t, x), \sigma^\top(t, x) \nabla u(t, x)) &= 0, \\ u(T, x) &= g(x). \end{aligned}$$

We call μ the drift, σ the diffusion coefficient or volatility, f the generator or driver and g the terminal condition. The involved PDE is called *semilinear parabolic* PDE.

The equation is parabolic through the second order diffusion part, and semilinear because the highest order derivatives enter linearly while u and ∇u may enter nonlinearly through f . For example, the Allen-Cahn equation of Section 5.2,

$$\partial_t u(t, x) = \Delta u(t, x) + u(t, x) - u(t, x)^3,$$

is a semilinear parabolic PDE. We calculate, for $\sigma = \sqrt{2} I_d$,

$$\frac{1}{2} \text{Tr}(\sigma \sigma^\top \mathcal{H}_x u) = \frac{1}{2} \text{Tr}(2 I_d \mathcal{H}_x u) = \sum_{i=1}^d \frac{\partial^2 u}{\partial x_i^2} = \Delta u,$$

and choose the parameters

$$\mu = 0, \quad \sigma = \sqrt{2} I_d, \quad f(t, x, y, z) = y - y^3.$$

A *backward stochastic differential equation* (BSDE) can be described by a terminal random variable ξ and a generator f such that a solution is a pair of adapted processes $(Y_t, Z_t)_{t \in [0, T]}$ satisfying

$$Y_t = \xi + \int_t^T f(s, Y_s, Z_s) ds - \int_t^T Z_s^\top dW_s.$$

Existence and uniqueness of this pair, together with the connection to partial differential equations, go back to [3].

3 The Deep BSDE Method

We want to compute the value $u(0, \xi)$ of the solution of a semilinear parabolic terminal value problem (Definition 2.2) in high dimension d . The main idea is to introduce a system consisting of a forward and a backward stochastic differential equation

$$\begin{aligned} X_t &= \xi + \int_0^t \mu(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s, \\ Y_t &= g(X_T) + \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s^\top dW_s. \end{aligned}$$

In differential notation, this system becomes

$$\begin{aligned} dX_t &= \mu(t, X_t) dt + \sigma(t, X_t) dW_t, & X_0 &= \xi, \\ dY_t &= -f(t, X_t, Y_t, Z_t) dt + Z_t^\top dW_t, & Y_T &= g(X_T). \end{aligned} \quad (1)$$

The following theorem draws the connection with the corresponding PDE.

Theorem 3.1: Generalized Feynman–Kac Formula

Let f be globally Lipschitz continuous in (x, y, z) and let g be globally Lipschitz continuous. Suppose that both functions have at most linear growth. Let

$$u \in C([0, T] \times \mathbb{R}^d) \cap C^{1,2}([0, T] \times \mathbb{R}^d)$$

solve the problem in Definition 2.2. Assume, in addition, that u and $\sigma^\top \nabla_x u$ have at most linear growth in the spatial variable. Then

$$Y_t := u(t, X_t), \quad Z_t := \sigma^\top(t, X_t) \nabla_x u(t, X_t), \quad t \in [0, T], \quad (2)$$

form the unique solution of the backward equation in (1).

Proof. See [4]. □

Consequently, instead of solving the PDE directly, we may solve the corresponding forward–backward stochastic differential equation (FBSDE). Solving (1) forward in time requires the initial values Y_0 and Z_0 , which are the unknown. On a grid $0 = t_0 < t_1 < \dots < t_N = T$ with $\Delta t_n = t_{n+1} - t_n$ and $\Delta W_n = W_{t_{n+1}} - W_{t_n}$ we discretize (1) by the Euler scheme

$$\begin{aligned} X_{t_{n+1}} &= X_{t_n} + \mu(t_n, X_{t_n}) \Delta t_n + \sigma(t_n, X_{t_n}) \Delta W_n, \\ Y_{t_{n+1}} &= Y_{t_n} - f(t_n, X_{t_n}, Y_{t_n}, Z_{t_n}) \Delta t_n + Z_{t_n}^\top \Delta W_n, \quad n = 0, \dots, N-1. \end{aligned} \quad (3)$$

We make (3) computable by three substitutions. The unknown initial value Y_{t_0} becomes a trainable parameter $\theta_{u_0} \in \mathbb{R}$, the unknown initial gradient Z_{t_0} becomes a trainable parameter $\theta_{\nabla u_0} \in \mathbb{R}^d$, and the process Z is approximated on the grid by feedforward neural networks $\varphi^{\theta_n} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with parameters θ_n . The scheme becomes

$$\begin{aligned} X_{t_{n+1}} &= X_{t_n} + \mu(t_n, X_{t_n}) \Delta t_n + \sigma(t_n, X_{t_n}) \Delta W_n, \\ Y_{t_{n+1}}^\theta &= Y_{t_n}^\theta - f(t_n, X_{t_n}, Y_{t_n}^\theta, Z_{t_n}^\theta) \Delta t_n + (Z_{t_n}^\theta)^\top \Delta W_n, \end{aligned} \quad (4)$$

with

$$Y_{t_0}^\theta = \theta_{u_0}, \quad Z_{t_0}^\theta = \theta_{\nabla u_0}, \quad Z_{t_n}^\theta = \varphi^{\theta_n}(X_{t_n}), \quad n = 1, \dots, N-1, \quad (5)$$

and the full parameter vector is

$$\theta = (\theta_{u_0}, \theta_{\nabla u_0}, \theta_1, \dots, \theta_{N-1}). \quad (6)$$

The terminal value $Y_{t_N}^\theta$ of (4) is an estimate of $g(X_{t_N})$, and we train all parameters jointly on the expected

terminal loss

$$\ell(\theta) = \mathbb{E} \left[|g(X_{t_N}) - Y_{t_N}^\theta|^2 \right].$$

The expectation is taken with respect to the underlying probability measure $\mathbb{P}$, that is, over the Brownian sample paths. In practice, we replace it by the empirical average over M independent simulated trajectories. The deep BSDE estimator of the solution value at the point $(0, \xi)$ is the trained initial parameter,

$$\hat{u}_{\text{NN}}(0, \xi) := \theta_{u_0}^*, \quad \theta^* \in \arg \min_{\theta} \frac{1}{M} \sum_{m=1}^M \left| g(X_{t_N}^{(m)}) - Y_{t_N}^{\theta, (m)} \right|^2, \quad (7)$$

where M is the number of simulated paths. All numerical results below compare $\hat{u}_{\text{NN}}(0, \xi)$ with reference values for $u(0, \xi)$.

4 Implementation

We use our own implementation of the deep BSDE solver inspired by the author's implementation [2], but allow to test for different network structures. For each $n = 1, \dots, N-1$, the network φ^{θ_n} has H hidden layers. Setting

$$h_n^0 := X_{t_n},$$

the output of its ℓ -th hidden layer is defined recursively by

$$h_n^\ell := \rho(A_n^\ell h_n^{\ell-1} + b_n^\ell), \quad \ell = 1, \dots, H,$$

where A_n^ℓ and b_n^ℓ denote the weight matrix and bias vector of the ℓ -th layer, respectively, and ρ is an activation function applied componentwise. The output layer is given by

$$\varphi^{\theta_n}(X_{t_n}) := A_n^{H+1} h_n^H + b_n^{H+1},$$

where

$$\theta_n := ((A_n^\ell, b_n^\ell))_{\ell=1}^{H+1}$$

collects all trainable parameters of the network. Consequently,

$$Z_{t_n}^\theta = \varphi^{\theta_n}(X_{t_n}), \quad n = 1, \dots, N-1.$$

Furthermore we aim to apply the code to financial problems in later works. Each equation enters as a PDE object holding the coefficients of Definition 2.2, the drift, the diffusion, the generator and the terminal condition. The network we used for results in this documentation consists of subnetworks φ^{θ_n} and has one d -dimensional input layer, two hidden layers of width 110 with tanh activation and no batch normalization, and a d -dimensional linear output layer. We optimize with Adam under a decaying learning rate. The initial value θ_{u_0} starts at a problem-specific guess, according to the initialization range of the paper. From there we iterate forward, taking into account the neural network approximations of Z , generated diffusion X and Brownian motion increments dW_t in each timestep, see Figure 2.

The used parameters are given in the Tables 1, 2, 3 and 4. The notebook `examples.ipynb` guides

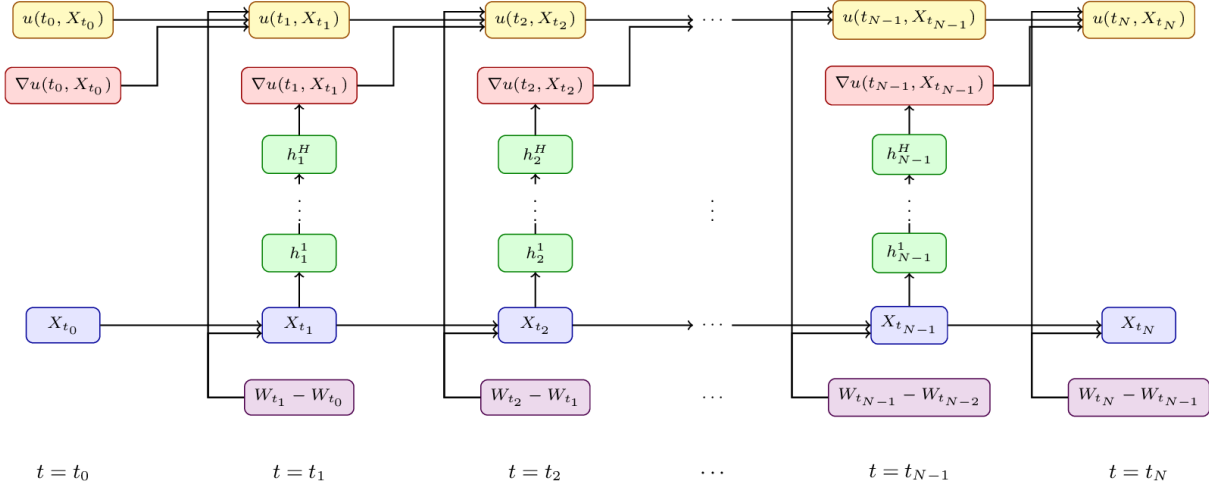


Figure 2: Unrolled architecture of the deep BSDE solver, one column per time step, illustration after [1].

through the setup and produces all results presented in this documentation.

5 Examples

In this Section we study the examples presented in the main article [1].

5.1 Hamilton-Jacobi-Bellman Equation

Given is the stochastic control problem

$$\inf_{\{m_t\}_{t \in [0, T]}} \mathbb{E} \left[\int_0^T \|m_t\|^2 dt + g(X_T) \right],$$

$$dX_t = 2\sqrt{\lambda} m_t dt + \sqrt{2} dW_t, \quad X_0 = x,$$

where $\lambda > 0$, $X = (X_t)_{t \in [0, T]}$ and $m = (m_t)_{t \in [0, T]}$ are $\mathbb{R}^{100}$ -valued processes, and $W = (W_t)_{t \in [0, T]}$ is a 100-dimensional Brownian motion. The terminal condition is

$$g(x) = \ln\left(\frac{1}{2} (1 + \|x\|^2)\right).$$

One can show, that the value function

$$u(t, x) := \inf_{\{m_t\}_{t \in [0, T]}} \mathbb{E} \left[\int_t^T \|m_s\|^2 ds + g(X_T) \middle| X_t = x \right]$$

is the solution of the Hamilton-Jacobi-Bellman equation

$$\partial_t u(t, x) + \Delta u(t, x) + \min_{m \in \mathbb{R}^{100}} \left(2\sqrt{\lambda} m \cdot \nabla u(t, x) + \|m\|^2 \right) = 0, \quad u(T, x) = g(x). \quad (8)$$

To evaluate the minimum in (8) we introduce, for fixed (t, x) , the function

$$F(m) := 2\sqrt{\lambda} m \cdot \nabla u(t, x) + \|m\|^2, \quad m \in \mathbb{R}^{100}, \quad (9)$$

and compute

$$\begin{aligned} \nabla F(m) &= 2\sqrt{\lambda} \nabla u(t, x) + 2m, \\ \mathcal{H} F(m) &= 2 I_d. \end{aligned}$$

The Hessian is positive definite, so F is strictly convex and its unique stationary point is the global minimum. We solve the first order condition and insert the minimizer into (9),

$$\begin{aligned} 0 &= \nabla F(m^*) = 2\sqrt{\lambda} \nabla u(t, x) + 2m^*, \\ m^* &= -\sqrt{\lambda} \nabla u(t, x), \\ F(m^*) &= -2\lambda \|\nabla u(t, x)\|^2 + \lambda \|\nabla u(t, x)\|^2 = -\lambda \|\nabla u(t, x)\|^2. \end{aligned} \quad (10)$$

Inserting (10) into (8) removes the optimization and leaves the reduced form,

$$\partial_t u(t, x) + \Delta u(t, x) - \lambda \|\nabla u(t, x)\|^2 = 0, \quad u(T, x) = g(x), \quad (11)$$

which is a semilinear parabolic terminal value problem with

$$\mu = 0, \quad \sigma = \sqrt{2} I_d, \quad f(t, x, y, z) = -\frac{\lambda}{2} \|z\|^2.$$

With the transformation

$$w := e^{-\lambda u} \quad (12)$$

we compute

$$\begin{aligned} \partial_t w &= -\lambda w \partial_t u, \\ \nabla w &= -\lambda w \nabla u, \\ \Delta w &= \nabla \cdot (-\lambda w \nabla u) = -\lambda w \Delta u + \lambda^2 w \|\nabla u\|^2, \end{aligned} \quad (13)$$

hence, by (11), we get

$$\partial_t w + \Delta w = -\lambda w (\partial_t u + \Delta u - \lambda \|\nabla u\|^2) = 0, \quad w(T, x) = e^{-\lambda g(x)}. \quad (14)$$

The Feynman-Kac formula 2.1 therefore yields

$$w(t, x) = \mathbb{E} \left[\exp \left(-\lambda g \left(x + \sqrt{2} W_{T-t} \right) \right) \right], \quad (15)$$

and solving for u we obtain the formula stated in the paper,

$$u(t, x) = -\frac{1}{\lambda} \ln \left(\mathbb{E} \left[\exp \left(-\lambda g \left(x + \sqrt{2} W_{T-t} \right) \right) \right] \right). \quad (16)$$

We estimate (16) at $(0, x)$ by the empirical mean over M_{MC} independent samples $W_T^{(1)}, \dots, W_T^{(M_{\text{MC}})} \sim \mathcal{N}(0, T I_d)$,

$$\hat{u}_{\text{MC}}(0, x) = -\frac{1}{\lambda} \ln \left(\frac{1}{M_{\text{MC}}} \sum_{m=1}^{M_{\text{MC}}} \exp \left(-\lambda g \left(x + \sqrt{2} W_T^{(m)} \right) \right) \right),$$

and compare it with the deep BSDE estimator $\hat{u}_{\text{NN}}(0, x)$ of (7). For $\lambda = 1$ we obtain

$$\hat{u}_{\text{MC}}(0, 0) = 4.5903. \quad (17)$$

We apply the generalized Feynman-Kac formula (2) on the PDE and get the system

$$\begin{aligned} X_t &= x + \sqrt{2} W_t, \\ dY_t &= \frac{\lambda}{2} \|Z_t\|^2 dt + Z_t^\top dW_t, \quad Y_T = g(X_T), \end{aligned}$$

in which

$$Y_t = u(t, X_t), \quad Z_t = \sqrt{2} \nabla u(t, X_t). \quad (18)$$

We specified the parameters according to Table 1.

d	100	time steps N	20
T	1	learning rate	$3 \cdot 10^{-3}$, decaying to $3 \cdot 10^{-4}$
λ	1, 10, 20, 30, 40, 50	iterations	2500
x	$(0, \dots, 0)$	batch size	128
M_{MC}	$2 \cdot 10^6$	start of θ_{u_0}	$\mathbb{E}[g(X_T)]$, the $\lambda = 0$ value

Table 1: *Parameters of the HJB example.*

Figure 3 (left) shows the relative approximation error at the point $(0, (0, \dots, 0))$ for $\lambda = 1$ against the number of iteration steps. We obtain

$$\frac{|\hat{u}_{\text{NN}} - \hat{u}_{\text{MC}}|}{\hat{u}_{\text{MC}}} = 0.01 \%, \quad (19)$$

Figure 3 shows the development of the approximation error together with the results for each run with both methods. The maximal deviation over these values is 1.20% at $\lambda = 50$ and lies relatively close to the Monte Carlo approximation. We can therefore see, that the results from the paper are reproducible and show similar deviations. The initialization of the parameters will be discussed later on in a separate paragraph.

5.2 Allen-Cahn Equation

We analyze an Allen-Cahn equation in 100 dimensions, given by the initial value problem

$$\partial_t u(t, x) = \Delta u(t, x) + u(t, x) - u(t, x)^3, \quad u(0, x) = g(x) = \frac{1}{2 + 0.4 \|x\|^2}. \quad (20)$$

No closed-form solution is available. The comparison value

$$u(0.3, (0, \dots, 0)) \approx 0.0528 \quad (21)$$

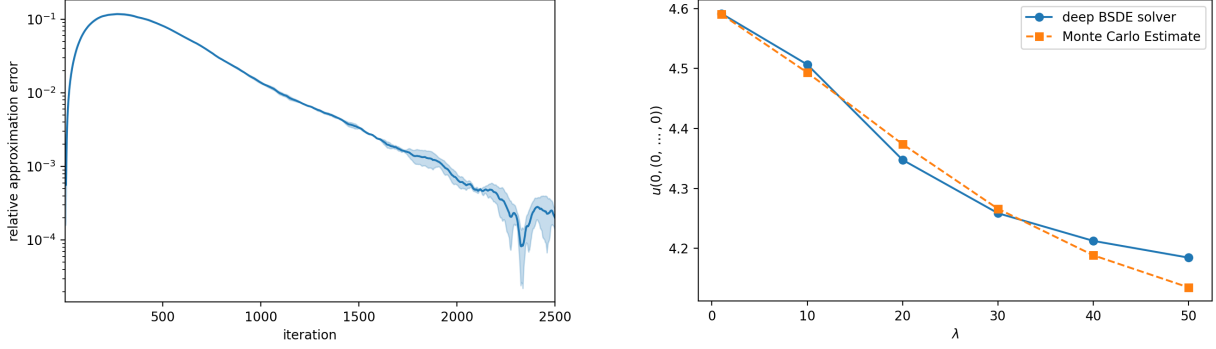


Figure 3: Relative error at $\lambda = 1$ (left) and $u(0, (0, \dots, 0))$ for $\lambda \in \{1, 10, 20, 30, 40, 50\}$, compare Fig. 2 of [1].

was computed with the branching-diffusion method in the paper. The deep BSDE method expects a semilinear parabolic terminal value problem, so we reverse time. We set

$$v(t, x) := u(T - t, x), \quad (t, x) \in [0, T] \times \mathbb{R}^{100}, \quad (22)$$

and compute, by the chain rule,

$$\begin{aligned} \partial_t v(t, x) &= -(\partial_t u)(T - t, x), \\ \Delta v(t, x) &= (\Delta u)(T - t, x), \\ v(T, x) &= u(0, x) = g(x). \end{aligned} \quad (23)$$

Evaluating (20) at the time point $T - t$ and substituting gives

$$\begin{aligned} -\partial_t v(t, x) &= \Delta v(t, x) + v(t, x) - v(t, x)^3, \\ 0 &= \partial_t v(t, x) + \Delta v(t, x) + v(t, x) - v(t, x)^3, \quad v(T, x) = g(x), \end{aligned}$$

which is the desired form. The corresponding forward-backward system reads

$$\begin{aligned} X_t &= \sqrt{2} W_t, \\ dY_t &= -(Y_t - Y_t^3) dt + Z_t^\top dW_t, \quad Y_T = g(X_T), \end{aligned} \quad (24)$$

with

$$Y_t = v(t, X_t), \quad Z_t = \sqrt{2} \nabla v(t, X_t). \quad (25)$$

The deep BSDE solver applied to (24) approximates

$$v(0, 0) = u(T, (0, \dots, 0)), \quad (26)$$

so training with horizon T returns the value of the original solution at time T . We specified the parameters according to Table 2. Figure 4 (left) shows the relative error of $\hat{u}_{\text{NN}}$ at $t = 0.3$ against the number of

d	100	time steps N	20
T	0.3	learning rate	10^{-3} , decaying to 10^{-4}
x	$(0, \dots, 0)$	iterations	4000
		batch size	64

Table 2: *Parameters of the Allen-Cahn example.*

iteration steps. We obtain

$$\hat{u}_{\text{NN}} = 0.0528, \quad \frac{|\hat{u}_{\text{NN}} - 0.0528|}{0.0528} = 0.02\%, \quad (27)$$

against 0.30 % in the paper. Figure 4 (right) shows the time evolution of $u(t, (0, \dots, 0))$, obtained by repeating the training for the times $t \in \{0.02, 0.05, 0.08, 0.11, 0.14, 0.18, 0.22, 0.26, 0.30\}$.

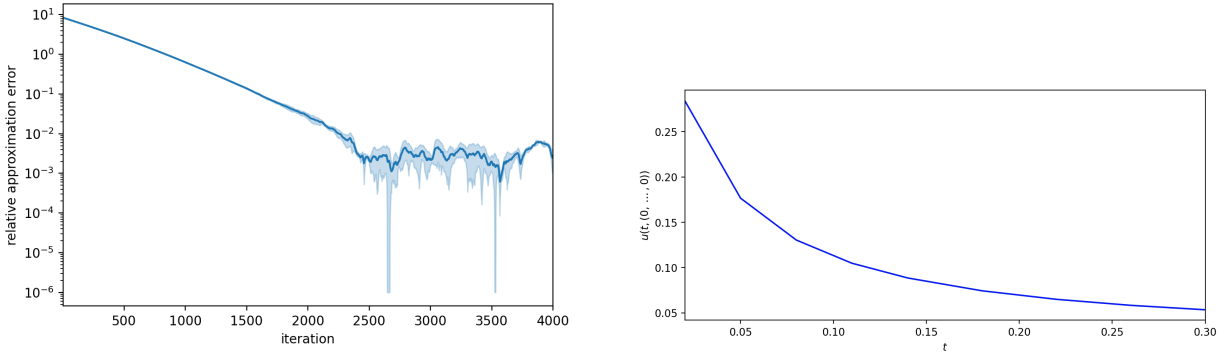


Figure 4: *Allen-Cahn equation, relative error at $T = 0.3$ (left) and time evolution of $u(t, (0, \dots, 0))$ (right), compare Fig. 3 of [1].*

5.3 Nonlinear Black-Scholes Equation with Default Risk

We start from the classical Black-Scholes equation. For a single underlying with volatility $\bar{\sigma}$ and risk-free rate r , the arbitrage-free price $u(t, x)$ of a European derivative with payoff g solves the linear terminal value problem

$$\partial_t u(t, x) + r x \partial_x u(t, x) + \frac{\bar{\sigma}^2}{2} x^2 \partial_{xx}^2 u(t, x) - r u(t, x) = 0, \quad u(T, x) = g(x), \quad (28)$$

where x is the current price of the underlying. The paper augments (28) in two directions. First, the claim depends on 100 underlyings following componentwise geometric Brownian motion,

$$dX_t^i = \bar{\mu} X_t^i dt + \bar{\sigma} X_t^i dW_t^i, \quad i = 1, \dots, 100,$$

and pays the minimum of the underlyings at maturity,

$$g(x) = \min(x_1, \dots, x_{100}).$$

Second, the issuer may default. Default is modeled as the first jump of a Poisson process whose intensity $Q(u)$ is a decreasing function of the current value u , default becomes more likely when the claim is worth

less. The intensity is piecewise linear with three regions,

$$Q(y) = \mathbf{1}_{(-\infty, v^h)}(y) \gamma^h + \mathbf{1}_{[v^l, \infty)}(y) \gamma^l + \mathbf{1}_{[v^h, v^l)}(y) \left[\frac{\gamma^h - \gamma^l}{v^h - v^l} (y - v^h) + \gamma^h \right]. \quad (29)$$

The three parts of (29) correspond to three value regimes. If the current value y lies below the lower

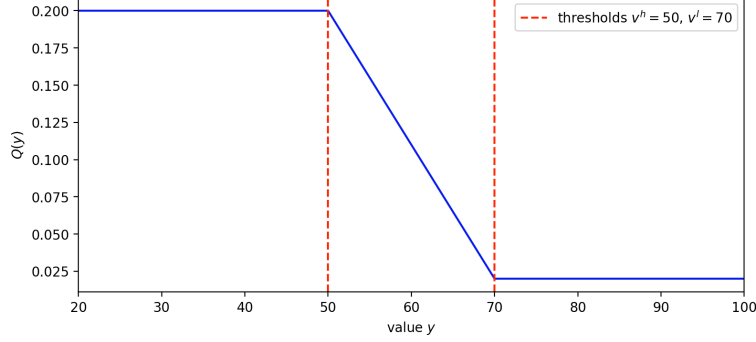


Figure 5: The default intensity Q of (29).

threshold v^h , the default intensity takes the high constant level γ^h , a claim with high default probability. If y lies above the upper threshold v^l , the intensity takes the low constant level γ^l , so the default probability is low. Between the thresholds the Q is linearly interpolated between the two levels. Figure 5 shows its graph. The parameters γ^h and γ^l are therefore the default intensities in the high and low risk regimes. At default the holder recovers only the fraction $\delta \in [0, 1)$ of the current value. The default rate enters the equation via the term

$$(1 - \delta) Q(u) u, \quad (30)$$

additionally with a discounting component Ru . The extended PDE takes the form

$$\partial_t u + \bar{\mu} x \cdot \nabla u + \frac{\bar{\sigma}^2}{2} \sum_{i=1}^{100} |x_i|^2 \frac{\partial^2 u}{\partial x_i^2} - (1 - \delta) Q(u) u - Ru = 0, \quad u(T, x) = g(x),$$

which is a semilinear parabolic terminal value problem with

$$\mu(x) = \bar{\mu} x, \quad \sigma(x) = \bar{\sigma} \text{diag}(x), \quad f(t, x, y, z) = -(1 - \delta) Q(y) y - Ry.$$

Here, again no closed-form solution is available. The comparison value

$$u(0, (100, \dots, 100)) \approx 57.300 \quad (31)$$

was computed with the multilevel Picard method [6], as quoted in the paper. The backward equation in this case reads

$$dY_t = [(1 - \delta) Q(Y_t) + R] Y_t dt + Z_t^\top dW_t, \quad Y_T = \min(X_T^1, \dots, X_T^{100}),$$

with

$$Y_t = u(t, X_t), \quad Z_t = \bar{\sigma} \text{diag}(X_t) \nabla u(t, X_t) \quad (32)$$

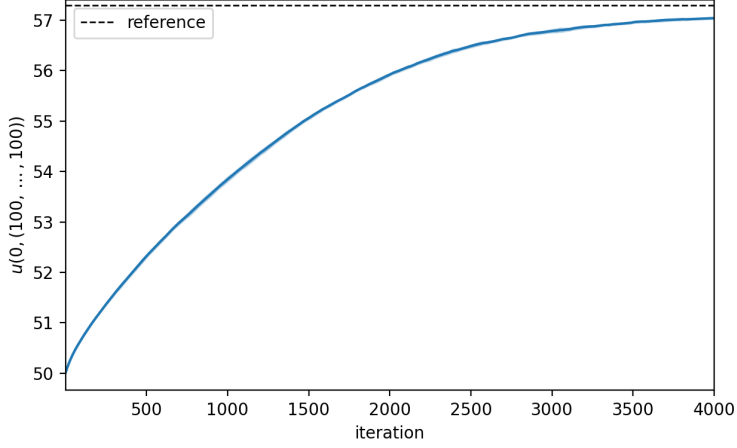


Figure 6: Default risk, approximation of $u(0, (100, \dots, 100))$, compare Fig. 1 of [1].

with parameters in Table 3. Figure 6 shows the approximation of $u(0, (100, \dots, 100))$ against the number

d	100	δ	2/3	time steps N	40
T	1	R	0.02	learning rate	10^{-2} , decaying to 10^{-3}
$\bar{\mu}$	0.02	v^h, v^l	50, 70	iterations	4000
$\bar{\sigma}$	0.2	γ^h, γ^l	0.2, 0.02	batch size	64
X_0	(100, ..., 100)	start of θ_{u_0}	50		

Table 3: Parameters of the default-risk example.

of iteration steps. We obtain an estimate and error of

$$\hat{u}_{\text{NN}} = 57.042, \quad \frac{|\hat{u}_{\text{NN}} - 57.300|}{57.300} = 0.45\%, \quad (33)$$

against 0.46% in the paper.

5.4 Reaction-Diffusion Equation

The last example of the paper is the time-dependent reaction-diffusion equation

$$\partial_t u(t, x) + \frac{1}{2} \Delta u(t, x) + \min \left\{ 1, (u(t, x) - u^*(t, x))^2 \right\} = 0, \quad u(T, x) = u^*(T, x), \quad (34)$$

with the function

$$u^*(t, x) = 1 + \kappa + \sin \left(\lambda \sum_{i=1}^d x_i \right) \exp \left(\frac{\lambda^2 d (t - T)}{2} \right), \quad \kappa = 0.6, \quad \lambda = \frac{1}{\sqrt{d}}.$$

The function u^* itself solves (34). Writing $s(x) := \sin(\lambda \sum_i x_i)$ and $e(t) := \exp(\lambda^2 d (t - T)/2)$ we compute

$$\begin{aligned} \partial_t u^*(t, x) &= \frac{\lambda^2 d}{2} s(x) e(t), \\ \Delta u^*(t, x) &= -\lambda^2 d s(x) e(t), \end{aligned} \quad (35)$$

so $\partial_t u^* + \frac{1}{2} \Delta u^* = 0$, the reaction term vanishes at $u = u^*$, and the terminal condition holds by construction. The exact value at the origin is

$$u(0, (0, \dots, 0)) = 1 + \kappa = 1.6. \quad (36)$$

Equation (34) is a semilinear parabolic terminal value problem with

$$\mu = 0, \quad \sigma = I_d, \quad f(t, x, y, z) = \min \left\{ 1, (y - u^*(t, x))^2 \right\}. \quad (37)$$

In the paper, it got solved with 0 to 4 hidden layers per subnetwork and 40000 iterations and reports that the relative error decreases in the number of hidden layers. We reconstruct the study for 1 to 4 hidden layers at the 40000 epochs of training of the paper. We specified the parameters according to Table 4. Since the terminal condition at $t = 0$ already is the exact value, we start θ_{u_0} away from the solution at 0.5.

d	100	time steps N	30
T	1	learning rate	10^{-2} , decaying to $3 \cdot 10^{-4}$
κ	0.6	subnetwork rate factor	0.1
λ	0.1	iterations	40000
x	$(0, \dots, 0)$	batch size	64
hidden layers	1, 2, 3, 4	start of θ_{u_0}	0.5

Table 4: *Parameters of the reaction-diffusion example.*

Table 5 and Figure 7 compare the relative errors per network depth with Table 1 of the paper, at the 40000 iterations of the paper. The error decreases in the number of hidden layers. At four hidden layers it reaches 0.93 % against 0.53 % in the paper, the first two depths end on the same level near 2.5 %, and only the deeper networks push below it.

hidden layers	1	2	3	4
this reconstruction	2.49 %	2.48 %	1.72 %	0.93 %
paper	0.90 %	0.60 %	0.56 %	0.53 %

Table 5: *Relative error per number of hidden layers.*

5.5 Stability in the Starting Value

Each example is trained again from four starting values of the trainable initial value $\widehat{Y}_0$, one seed per value and all other settings unchanged. Figure 8 collects the four runs, panel (a) shows the HJB equation, (b) the Allen-Cahn equation, (c) the default risk equation and (d) the reaction-diffusion equation. Every curve moves toward the reference, and no run diverges or settles at a wrong level. The further away the start is, the more iterations the curve needs to get there. The starts furthest away in the first two equations have not fully arrived after 5000 iterations, while in the default risk example 10000 iterations bring even the starts of 20 and 90 close to the reference, although they begin deep in the high and low risk regime of Q . The reaction-diffusion runs collapse onto a common path within a few hundred iterations and settle together slightly above the benchmark, so the remaining error stems from the method and not from the initialization.

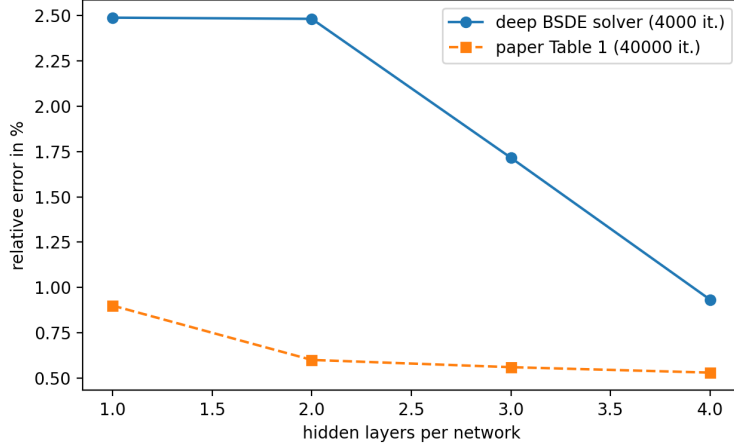


Figure 7: Relative error against the number of hidden layers, compare Table 1 of [1].

6 Conclusion

We could replicate all four examples of [1]. The errors match the accuracy reported in the paper, and the hidden-layer study reproduces the trend of the paper’s Table 1 and comes close to its level. All runs together take a few hours on a laptop CPU. We have made the following observations.

- (1) The choice of the starting value can have an effect on how long effective training takes, but the optimization was successful for all tested initializations, see Section 5.5.
- (2) The accuracy of the examples could often be reached, sometimes even improved.
- (3) The 100-dimensional examples do not need substantially more time than lower dimensional ones, so we have verified that the curse of dimensionality can indeed be overcome by this method.

References

- [1] J. Han, A. Jentzen, W. E, *Solving high-dimensional partial differential equations using deep learning*, Proceedings of the National Academy of Sciences 115(34), 8505–8510, 2018.
- [2] J. Han, *DeepBSDE, Deep BSDE solver in TensorFlow*, <https://github.com/frankhan91/DeepBSDE>.
- [3] E. Pardoux, S. Peng, *Adapted solution of a backward stochastic differential equation*, Systems & Control Letters 14(1), 55–61, 1990.
- [4] N. El Karoui, S. Peng, M. C. Quenez, *Backward stochastic differential equations in finance*, Mathematical Finance 7(1), 1–71, 1997.
- [5] B. Øksendal, *Stochastic Differential Equations, An Introduction with Applications*, 6th edition, Springer, Berlin, 2003.
- [6] W. E, M. Hutzenthaler, A. Jentzen, T. Kruse, *On multilevel Picard numerical approximations for high-dimensional nonlinear parabolic partial differential equations and high-dimensional nonlinear backward stochastic differential equations*, Journal of Scientific Computing 79(3), 1534–1571, 2019.

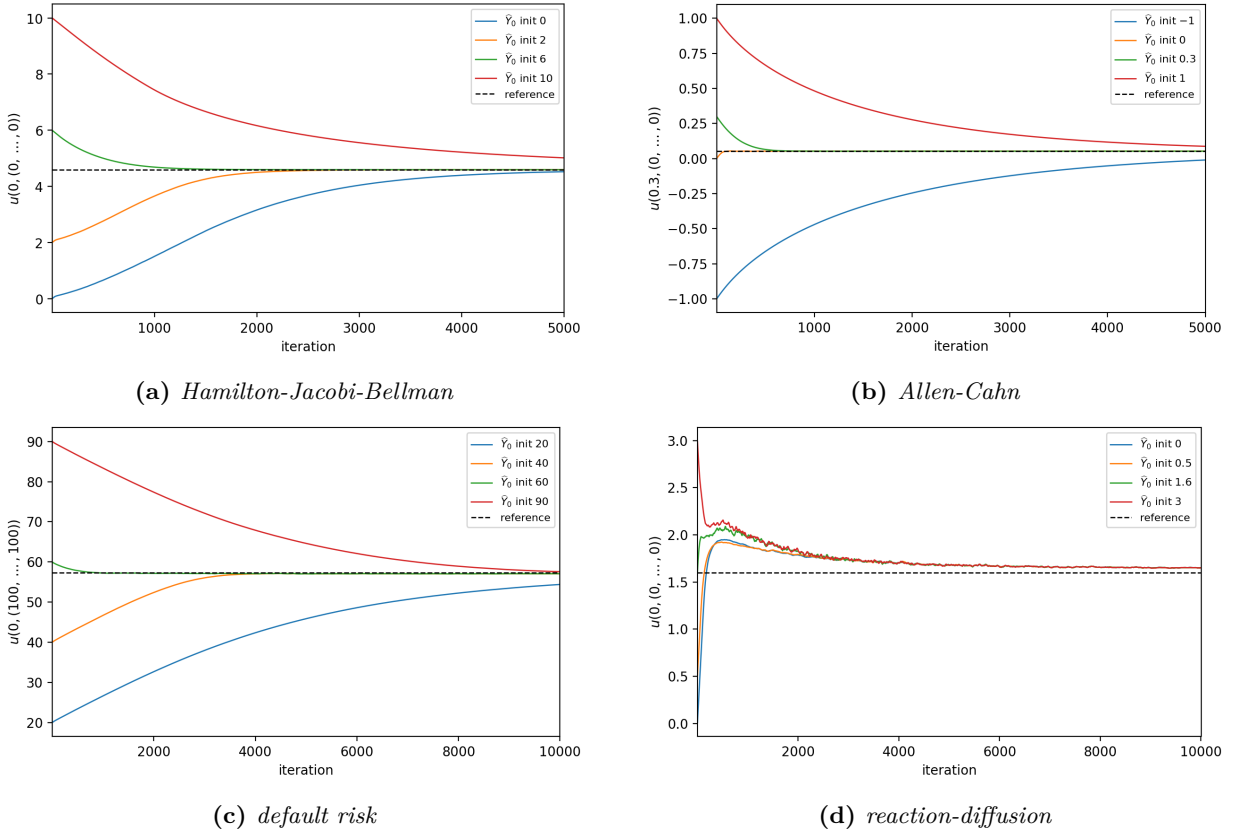


Figure 8: Training from four starting values of $\hat{Y}_0$ per example.

List of Figures

1	The three formulations of a parabolic terminal value problem and their numerical methods.	1
2	Unrolled architecture of the deep BSDE solver, one column per time step, illustration after [1].	7
3	Relative error at $\lambda = 1$ (left) and $u(0, (0, \dots, 0))$ for $\lambda \in \{1, 10, 20, 30, 40, 50\}$, compare Fig. 2 of [1].	9
4	Allen-Cahn equation, relative error at $T = 0.3$ (left) and time evolution of $u(t, (0, \dots, 0))$ (right), compare Fig. 3 of [1].	11
5	The default intensity Q of (29).	12
6	Default risk, approximation of $u(0, (100, \dots, 100))$, compare Fig. 1 of [1].	12
7	Relative error against the number of hidden layers, compare Table 1 of [1].	14
8	Training from four starting values of $\hat{Y}_0$ per example.	15